

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Mikroelektroniki i Optoelektroniki

Praca dyplomowa inżynierska

na kierunku Elektronika
w specjalności Elektronika i Fotonika

Dedykowana aplikacja do analizy analogowych układów elektronicznych
w języku programowania Swift

Oleksandr Bolbat

Numer albumu 329915

promotor
mgr inż. Bartosz Dec

WARSZAWA 2026

Streszczenie

Praca dotyczy projektu i implementacji prostego symulatora układów elektronicznych wykorzystującego metode Modified Nodal Analysis. Program przetwarza tekstowy opis obwodu, tworzy zbior stempli elementow, składa macierz układu i rozwiązuje punkt pracy dla układów liniowych.

Słowa kluczowe: Modified Nodal Analysis, symulacja obwodów, SPICE, Swift, metody numeryczne.

Abstract

The thesis describes the design and implementation of a simple electronic circuit simulator based on Modified Nodal Analysis. The program parses a textual circuit description, creates element stamps, assembles the system matrix, and solves the operating point for linear circuits.

Keywords: Modified Nodal Analysis, circuit simulation, SPICE, Swift, numerical methods.

Spis treści

1	Wstęp	1
1.1	Motywacja	1
1.2	Cel pracy	1
1.3	Zakres pracy	1
1.4	Struktura pracy	1
2	Podstawy teoretyczne	2
2.1	Opis układu elektronicznego	2
2.2	Analiza węzłów	2
2.3	Ograniczenia klasycznej analizy węzłów	2
2.4	Idea symulatorów typu SPICE	2
3	Modified Nodal Analysis	3
3.1	Postać układu równań	3
3.2	Stemple elementów	3
3.3	Elementy obsługiwane w programie	3
3.4	Komenda punktu pracy	4
4	Projekt programu	5
4.1	Architektura	5
4.2	Format pliku wejściowego	5
4.3	Reprezentacja elementów	5
4.4	Obsługa błędów	5
5	Implementacja	6
5.1	Parser	6
5.2	Stemple	6
5.3	Macierze i rozwiązywanie układu	6
5.4	Przykład przepływu danych	6
6	Testy	7
6.1	Zakres testów	7

6.2	Testy parsera	7
6.3	Testy solvera	7
6.4	Testy błędów stempli	7
7	Podsumowanie	8
7.1	Wnioski	8
7.2	Możliwe kierunki rozwoju	8
	Bibliografia	9

Rozdział 1

Wstęp

1.1 Motywacja

Symulatory układów elektronicznych pozwalają analizować zachowanie obwodów bez konieczności budowania fizycznego prototypu. Jednym z najważniejszych podejść stosowanych w takich narzędziach jest Modified Nodal Analysis, która rozszerza klasyczną metodę potencjałów węzłowych o dodatkowe niewiadome [1].

1.2 Cel pracy

Celem pracy jest zaprojektowanie i zaimplementowanie programu, który wczytuje tekstowy opis układu, przekształca elementy obwodu na stemple macierzowe oraz oblicza punkt pracy układu.

1.3 Zakres pracy

Zakres implementacji obejmuje parser plików tekstowych, model stempli elementów liniowych, składanie macierzy układu oraz solver dla komendy punktu pracy. Symulacja czasowa została przewidziana jako dalszy etap rozwoju programu.

1.4 Struktura pracy

W rozdział 2 przedstawiono podstawowe pojęcia teorii obwodów i symulacji. Rozdział 3 opisuje metodę Modified Nodal Analysis. Rozdział 4 i rozdział 5 omawiają architekturę oraz implementację programu. Rozdział 6 przedstawia sposób weryfikacji, a rozdział 7 zawiera wnioski i możliwe kierunki dalszego rozwoju.

Rozdział 2

Podstawy teoretyczne

2.1 Opis układu elektronicznego

Układ elektroniczny można opisać jako zbiór elementów połączonych przez węzły. Każdy element narzuca zależność pomiędzy napięciami i prądami. Dla elementów liniowych zależności te mogą zostać zapisane w postaci układu równań liniowych.

2.2 Analiza węzłów

Klasyczna analiza węzłów opiera się na prawie Kirchhoffa dla prądów. Niewiadomymi są potencjały węzłów względem węzła odniesienia. Metoda dobrze opisuje elementy, których prąd można wyrazić bezpośrednio przez napięcia węzłowe, na przykład rezystory i źródła prądowe.

2.3 Ograniczenia klasycznej analizy węzłów

Idealne źródła napięciowe oraz wybrane źródła sterowane nie zawsze mogą być wygodnie opisane za pomocą samych potencjałów węzłowych. W takich przypadkach do układu równań wprowadza się dodatkowe niewiadome, najczęściej prądy płynące przez źródła napięciowe.

2.4 Idea symulatorów typu SPICE

Symulatory typu SPICE rozbijają opis obwodu na lokalne wkłady elementów. Każdy element dodaje określone wartości do globalnej macierzy układu. Takie podejście pozwala oddzielić logikę elementów od procesu rozwiązywania równań [2].

Rozdział 3

Modified Nodal Analysis

3.1 Postać układu równan

W metodzie Modified Nodal Analysis układ równan można zapisać w postaci:

$$\begin{bmatrix} G & B \\ C & D \end{bmatrix} \begin{bmatrix} v \\ j \end{bmatrix} = \begin{bmatrix} i \\ e \end{bmatrix}. \quad (3.1)$$

Wektor v zawiera napięcia węzłowe, a wektor j dodatkowe niewiadome prądowe. Macierz G opisuje przewodności między węzłami, macierze B i C łączą węzły z dodatkowymi równaniami, a macierz D opisuje zależności pomiędzy dodatkowymi niewiadomymi.

3.2 Stemple elementów

Stempel elementu jest lokalnym wkładem do macierzy układu i wektora prawej strony. Dla rezystora o rezystancji R przewodność wynosi:

$$g = \frac{1}{R}. \quad (3.2)$$

Element podłączony pomiędzy węzłami n_1 i n_2 dodaje przewodność do pozycji diagonalnych oraz wartości przeciwne do pozycji pozadiagonalnych macierzy.

3.3 Elementy obsługiwane w programie

Implementacja obsługuje rezystor, niezależne źródło prądowe, niezależne źródło napięciowe oraz podstawowe źródła sterowane: CCCS, VCCS, CVCV i VCVS. Elementy wymagające dodatkowej niewiadomej zwiększają rozmiar macierzy układu.

3.4 Komenda punktu pracy

Komenda `.op` oznacza obliczenie punktu pracy. W obecnej wersji programu odpowiada ona zlozeniu macierzy dla opisu obwodu i rozwiazaniu otrzymanego ukkladu rownan liniowych.

Rozdział 4

Projekt programu

4.1 Architektura

Program został podzielony na kilka odpowiedzialności: parser, typy elementów, stemple, macierze oraz solver. Taki podział ułatwia rozszerzanie programu o kolejne elementy i kolejne rodzaje analizy.

4.2 Format pliku wejściowego

Opis układu ma postać tekstową. Każda linia elementu zaczyna się od nazwy typu, po której występują numery węzłów i parametry. Przykładowy opis może wyglądać następująco:

```
R 1 2 1000
DCCS 1 0 0.001
DCVS 2 0 5
.op
```

Listing 4.1: Przykładowy opis układu

4.3 Reprezentacja elementów

Każdy element jest reprezentowany przez typ, który tworzy odpowiedni stempel. Parser nie rozwiązuje układu samodzielnie, lecz przekształca opis tekstowy w dane wejściowe dla solvera.

4.4 Obsługa błędów

Błędy parsera dotyczą struktury pliku, na przykład nieznanego elementu, zbyt liczby parametrów albo braku komendy analizy. Błędy stempli dotyczą niepoprawnych parametrów elementu. Dzięki temu miejsce wykrycia błędu jest oddzielone od miejsca jego opisania.

Rozdział 5

Implementacja

5.1 Parser

Parser odczytuje plik tekstowy linia po linii, rozpoznaje typ elementu oraz komende analizy. Dla elementow wymagajacych dodatkowych niewiadomych obliczany jest numer nowego wiersza na podstawie liczby wezlow i liczby wczesniej dodanych dodatkowych rownan.

5.2 Stemple

Stempel zawiera macierz lokalna, wektor prawej strony oraz informacje o wierszach i kolumnach, do ktorych dane maja zostac dodane. Takie podejscie pozwala opisac rozne elementy przy pomocy wspolnego interfejsu.

5.3 Macierze i rozwiazywanie ukladu

Solver otrzymuje tablice stempli i komende analizy. Dla komendy `.op` tworzona jest globalna macierz ukladu, nastepnie wszystkie stemple sa do niej dodawane, a uklad liniowy jest rozwiazywany metoda eliminacji LU [3].

5.4 Przyklad przeplywu danych

Przeplyw danych w programie mozna opisac nastepujaco:

1. Plik tekstowy zostaje odczytany przez parser.
2. Parser tworzy stemple elementow i rozpoznaje komende analizy.
3. Solver sklada globalna macierz ukladu.
4. Solver zwraca macierz wynikowa dla punktu pracy.

Rozdział 6

Testy

6.1 Zakres testów

Testy jednostkowe obejmują parser, tworzenie stempli, obsługę błędów oraz solver. Osobne testy sprawdzają poprawne dane wejściowe i przypadki, w których program powinien zgłosić błąd.

6.2 Testy parsera

Parser jest testowany na pojedynczych elementach oraz na pliku zawierającym wszystkie obsługiwane typy elementów. Pozwala to sprawdzić, czy program poprawnie rozpoznaje elementy, parametry oraz komendy analizy.

6.3 Testy solvera

Testy solvera porównują wynik rozwiązania układu z oczekiwanymi wartościami dla prostych obwodów. Takie przypadki testowe są łatwe do zweryfikowania ręcznie i dobrze wykrywają błędy w składaniu macierzy.

6.4 Testy błędów stempli

Błędy parametrów stempli są sprawdzane oddzielnie od testów parsera i solvera. Dzięki temu testy pozostają czytelne i każdy zestaw sprawdza jedną warstwę programu.

Rozdział 7

Podsumowanie

7.1 Wnioski

Zaimplementowany program pokazuje pełny przepływ danych od tekstowego opisu układu do rozwiązania punktu pracy. Metoda Modified Nodal Analysis dobrze pasuje do takiej architektury, ponieważ pozwala każdemu elementowi dodawać lokalny wkład do globalnego układu równań.

7.2 Mozliwe kierunki rozwoju

Dalszy rozwój programu może obejmować symulacje czasowe dla komendy `.tran`, elementy nieliniowe, iteracje Newtona-Raphsona, bardziej rozbudowany format plików wejściowych oraz eksport wyników do plików.

Bibliografia

- [1] C.-W. Ho, A. E. Ruehli i P. A. Brennan, „The Modified Nodal Approach to Network Analysis”, *IEEE Transactions on Circuits and Systems*, t. 22, nr 6, s. 504–509, 1975. DOI: 10.1109/TCS.1975.1084079
- [2] L. O. Chua i P.-M. Lin, *Computer-Aided Analysis of Electronic Circuits: Algorithms and Computational Techniques*. Prentice-Hall, 1975.
- [3] A. Quarteroni, R. Sacco i F. Saleri, *Numerical Mathematics*, 2 wyd. Springer, 2007.